

Reviewer Pack — Minimal Rank of Tropicalized Sheaf Cohomology vs Monotone Ci...

Entry 6c0bedb7bb7d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 05:44:45 UTC

Minimal Rank of Tropicalized Sheaf Cohomology vs Monotone Circuit Depth for k-CLIQUE

Entry ID: 6c0bedb7bb7d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 05:44:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Tropical Geometry) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For any given k-CLIQUE instance, the minimal rank of its associated tropicalized sheaf cohomology groups is $\Theta(n^{\{1/4\}})$ ’, ‘The rank of the tropicalized sheaf cohomology groups for a k-CLIQUE instance scales polynomially with the number of vertices in the instance.’, ‘For all instances with $n \leq 40$, the ratio between the rank of the tropicalized sheaf cohomology and the vertex count is at least $\Theta(n^{\{-1/4\}})$.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a way to study complex structures through linear algebraic methods. Sheaf cohomology groups are fundamental in algebraic geometry, and their tropicalization might reveal hidden structural properties relevant to complexity theory.’, ‘The conjecture aims to exploit the geometric intuition from sheaf theory, which is rarely applied in complexity theory, to understand the structure of monotone circuits.’, ‘This connection could potentially lead to new insights into circuit lower bounds for k-CLIQUE and other monotone complexity problems.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8ccd1a5963299cfa

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The ratio of the minimal rank of tropicalized sheaf cohomology groups to the number of vertices in a k-CLIQUE instance is within $\Theta(n^{\{-1/4\}})$ for all instances with $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND monotone circuit complexity FOR k-CLIQUE AND minimal rank of sheaf cohomology - polynomial scaling IN tropicalized sheaf cohomology AND number of vertices IN k-CLIQUE instances - $\theta(n^{\{1/4\}})$ ratio BETWEEN sheaf cohomology rank AND vertex count IN k-CLIQUE

Top relevant hits considered: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/2309.17302v2] Geometry of tropical extensions of hyperfields - [http://arxiv.org/abs/2601.15320v1] On Brain as a Mathematical Manifold: Neural Manifolds, Sheaf Semantics, and Leibnizian Harmony - [http://arxiv.org/abs/2102.06927v3] A remark on singular cohomology and sheaf cohomology - [http://arxiv.org/abs/2406.00138v2] Mirror Symmetry and Level-rank Duality for 3d $\mathcal{N} = 4$ Rank 0 SCFTs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(n):
                if j != i:
                    factor = A[j][i] / A[i][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_rank(A):
        rref = gaussian_elimination([row[:] for row in A])
        rank = 0
```

```

    for row in rref:
        if any(row):
            rank += 1
    return rank

def k_clique_instance(n, k):
    edges = set()
    while len(edges) < k * (k - 1) // 2:
        u, v = random.sample(range(n), 2)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))
    return edges

n_values = [5, 10, 15, 20, 30, 40]
ratios = []

for n in n_values:
    for _ in range(5):
        clique = k_clique_instance(n, n // 2)
        adjacency_matrix = [[0] * n for _ in range(n)]
        for u, v in clique:
            adjacency_matrix[u][v] = 1
            adjacency_matrix[v][u] = 1

        rank = matrix_rank(adjacency_matrix)
        ratios.append(rank / n)

mean_ratio = sum(ratios) / len(ratios)
conjecture_holds = all(math.isclose(mean_ratio, n**(-0.25), rel_tol=1e-2) for n in n_values)

return {
    "metric_name": "Ratio of Rank to Vertex Count",
    "metric_value": mean_ratio,
    "instances_tested": len(ratios),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "mapping_undefined"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:

```

```

first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e141b519.py", line 80, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e141b519.py", line 60, in run_trial
    rank = matrix_rank(adjacency_matrix)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e141b519.py", line 34, in matrix_rank
    rref = gaussian_elimination([row[:] for row in A])
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e141b519.py", line 28, in gaussian_elimination
    factor = A[j][i] / A[i][i]
              ^^^^^^^
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's claim. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15180
2	propose	ollama_remote	glm4:latest	0	0	14104
3	preregistration	ollama_remote	glm4:latest	0	0	18811
4	novelty	ollama_remote	glm4:latest	0	0	11397
5	novelty	ollama_remote	glm4:latest	0	0	9252
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12218
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11405
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12334

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9469
10	judge	ollama_remote	glm4:latest	0	0	11503

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125675 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/6c0bedb7bb7d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/6c0bedb7bb7d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/6c0bedb7bb7d.tar.gz (if generated)